ACTIVIDAD 3 · CIBERSEGURIDAD

Seguridad en la nube, contenedores y APIs

Laboratorio de infraestructura como código, análisis de imágenes y autenticación JWT con verificación de seguridad.

Terraform · LocalStack · Docker · Trivy · JWT · OWASP ZAP



Alumno: Hugo Ruiz

Materia: Ciberseguridad

Fecha: noviembre de 2026

Repositorio:
github.com/Hugoatpxndx/actividad3-ciberseguridad

Resumen visual

Fase 1 · S3 seguro

3 recursos Terraform · AES-256 · 4/4 bloqueos públicos · ACL privada

Fase 2 · Contenedores

595 hallazgos y 23 críticas en `node:14` →
0 / 0
en `node:18-alpine`

Fase 3 · API + sitio

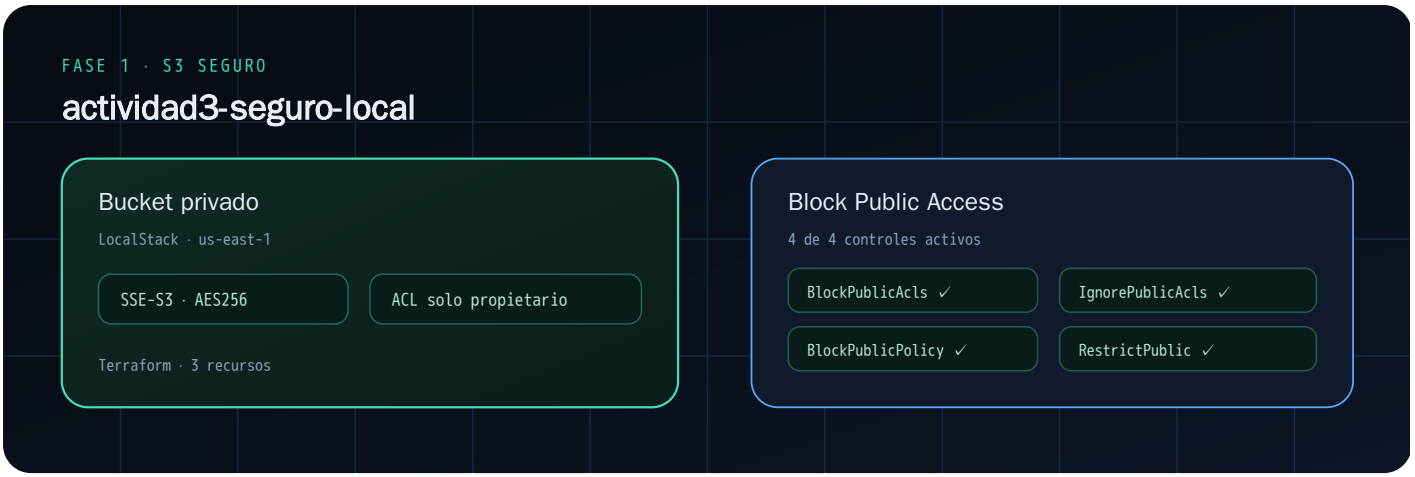
JWT HS256 · 401 sin token · 403 sin rol · 200 con admin · ZAP 0 FAIL

Fase	Tema	Herramientas	Resultado clave
1	Infraestructura como código	Terraform · LocalStack · AWS CLI	Bucket privado, AES-256, 4/4 bloqueos
2	Seguridad de contenedores	Docker · Trivy	595 hallazgos → 0
3	Desarrollo seguro y autenticación	Node.js · Express · JWT · OWASP ZAP	401 / 403 / 200 · 0 FAIL

Contenido

1. Resumen visual — resultados de las tres fases
2. Fase 1 · Bucket S3 seguro — Terraform, LocalStack y verificación con AWS CLI
3. Fase 2 · Seguridad de contenedores — Trivy, comparación de imágenes y Dockerfile final
4. Fase 3 · Sitio y API con JWT — login, matriz de autorización, ZAP e IAM
5. Conclusiones — aprendizajes y aplicación práctica
6. Referencias y anexos — fuentes y ubicación de las evidencias

Fase 1 · Bucket S3 seguro

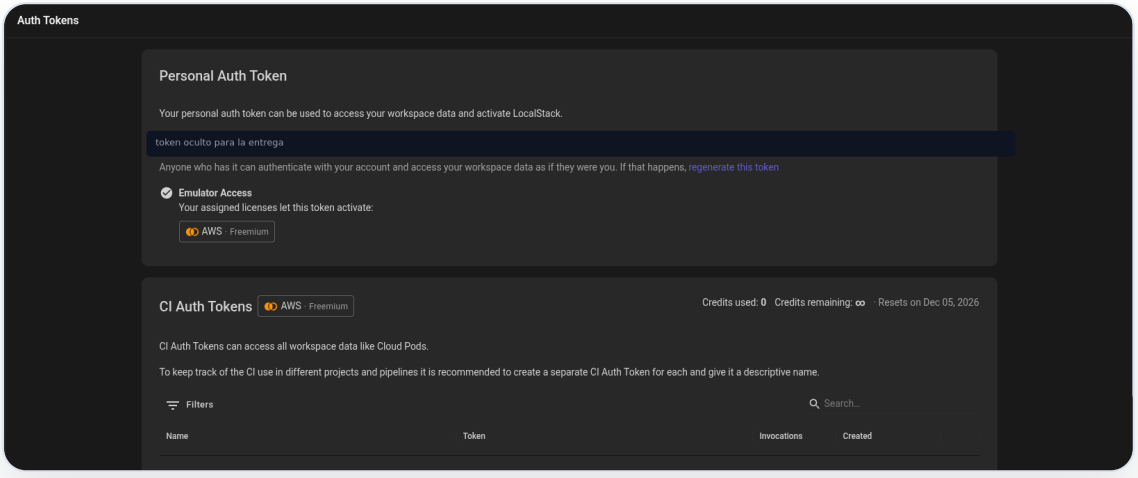


Entorno verificado

Herramienta	Versión	Estado
Docker Engine	29.9.3	OK
Docker Compose	5.7.0	OK
Terraform	1.15.9	OK
AWS CLI	2.37.9	OK
LocalStack	2026.10.6 (`pro` , plan Freemium — antes *Hobby*)	healthy · S3 available

Salida completa: `fase1/evidencias/versiones.txt` .

Plan de LocalStack



Plan gratuito. El panel muestra `AWS · Freemium` con créditos ilimitados: es el plan que la guía llama *Hobby — Free*. El token personal está oculto a propósito.

Recursos declarados

Recurso	Configuración
`aws_s3_bucket`	`actividad3-seguro-local`
`aws_s3_bucket_server_side_encryption_configuration`	`SSEAlgorithm = AES256`
`aws_s3_bucket_public_access_block`	4 controles en `true`

Verificación real

Consulta	Resultado
`terraform validate`	Success! The configuration is valid
`terraform apply`	3 added, 0 changed, 0 destroyed
`get-bucket-encryption`	"SSEAlgorithm": "AES256"
`get-public-access-block`	4/4 en true
`get-bucket-acl`	Solo propietario con FULL_CONTROL
`get-bucket-policy-status`	NoSuchBucketPolicy
Segundo `terraform plan`	No changes

> Lectura importante: `NoSuchBucketPolicy` solo indica que no hay una política de bucket adjunta. No significa que el bucket sea público. La privacidad se sostiene con la ACL privada y los cuatro bloqueos.

Plan de Terraform

terraform plan — 3 recursos · AES-256 · 4 bloqueos

```
$ terraform plan -out=tfplan

Terraform will perform the following actions:

  # aws_s3_bucket.secure_bucket will be created
    + bucket                      = "actividad3-seguro-local"

  # aws_s3_bucket_public_access_block.secure_bucket_pab will be created
    + block_public_acls       = true
    + block_public_policy     = true
    + ignore_public_acls      = true
    + restrict_public_buckets = true

  # aws_s3_bucket_server_side_encryption_configuration.secure_bucket_sse will be created
    + rule {
        + apply_server_side_encryption_by_default {
            + sse_algorithm     = "AES256"
          }
      }

Plan: 3 to add, 0 to change, 0 to destroy.
Saved the plan to: tfplan
```

terraform plan. Tres recursos declarados: bucket, bloqueo de acceso público y cifrado. Extracto de `fase1/evidencias/terraform-plan.txt`.

Verificación con AWS CLI

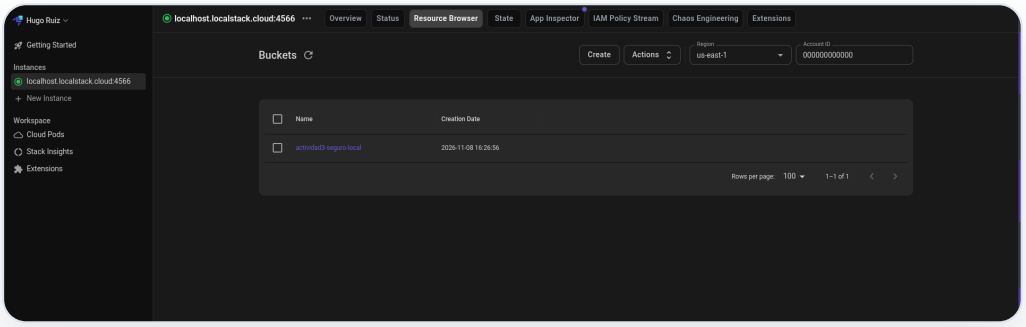
aws s3api · verificación — cifrado y bloqueo público reales

```
$ aws --endpoint-url=http://localhost:4566 s3api get-bucket-encryption --bucket actividad3-seguro-local
  "ServerSideEncryptionConfiguration": {
    "Rules": [
      {
        "ApplyServerSideEncryptionByDefault": {
          "SSEAlgorithm": "AES256"
        },
        "BucketKeyEnabled": false
      }
    ]
  }

$ aws --endpoint-url=http://localhost:4566 s3api get-public-access-block --bucket actividad3-seguro-local
  "PublicAccessBlockConfiguration": {
    "BlockPublicAcls": true,
    "IgnorePublicAcls": true,
    "BlockPublicPolicy": true,
    "RestrictPublicBuckets": true
  }
```

AWS CLI. El bucket responde `AES256` y los cuatro controles de bloqueo público en `true`. Extracto de `fase1/evidencias/verificacion-s3.txt`.

Dashboard con el bucket creado



LocalStack. El bucket `actividad3-seguro-local` visible en el panel web del emulador.

Carga del archivo de prueba

aws s3 · carga y listado — archivo de prueba dentro del bucket

```
$ aws --endpoint-url=http://localhost:4566 s3 cp archivo-prueba.txt s3://actividad3-seguro-local/
upload: ./archivo-prueba.txt to s3://actividad3-seguro-local/archivo-prueba.txt

$ aws --endpoint-url=http://localhost:4566 s3 ls s3://actividad3-seguro-local/
2026-11-08 16:27:12         36 archivo-prueba.txt
```

aws s3. El archivo de prueba se sube al bucket y aparece en el listado. Extracto de `fase1/evidencias/subida-archivo.txt`.

Acceso anónimo en LocalStack

```
HTTP/1.1 200 OK
x-amz-server-side-encryption: AES256
```

LocalStack permitió la lectura anónima. Es una limitación del simulador: no replica todas las reglas de autorización de AWS. La respuesta confirma que el objeto está cifrado con AES-256. En AWS real, con los cuatro bloqueos activos y sin política pública, la lectura anónima se deniega.

Conclusión de la fase

Nivel	Qué se comprobó
Infraestructura declarada	Terraform define el bucket, `AES256` y los cuatro bloqueos.
Configuración consultada	AWS CLI confirma cifrado, bloqueos y ACL privada.
Seguridad aplicada	En LocalStack la lectura anónima responde `200`; en AWS real se deniega.

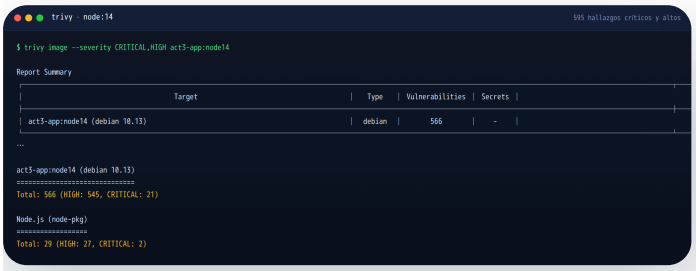
Fase 2 · Seguridad de contenedores



Imagen	SO	Node/npm	Total	Críticas	Altas
`act3-app:node14`	566	29	595	23	572
`act3-app:node18`	0	0	0	0	0

Alcance: los conteos corresponden a las severidades CRITICAL y HIGH (`trivy image --severity CRITICAL,HIGH`). No incluyen MEDIUM ni LOW.

Antes y después en la consola



node:14. 566 del sistema + 29 de paquetes: 23 críticas y 572 altas.



node:18-alpine. Sin hallazgos críticos ni altos tras las correcciones.

Salidas reales de Trivy 0.76.2, recortadas de `fase2/evidencias/trivy-node14.txt` y `trivy-node18.txt`.

Mitigaciones aplicadas

Medida	Efecto
`node:14` → `node:18-alpine`	Base más reciente y menor superficie
`apk upgrade --no-cache`	Corrige OpenSSL y zlib del sistema
Build multi-stage	Solo pasan a la imagen final las dependencias de producción

Medida	Efecto
Eliminar `npm`, `npx` y `corepack`	Quita del runtime paquetes vulnerables que la app no necesita
`USER node`	La aplicación no corre como root

Dockerfile final

```
FROM node:18-alpine AS deps
WORKDIR /app
COPY package*.json ./
RUN npm ci --omit=dev

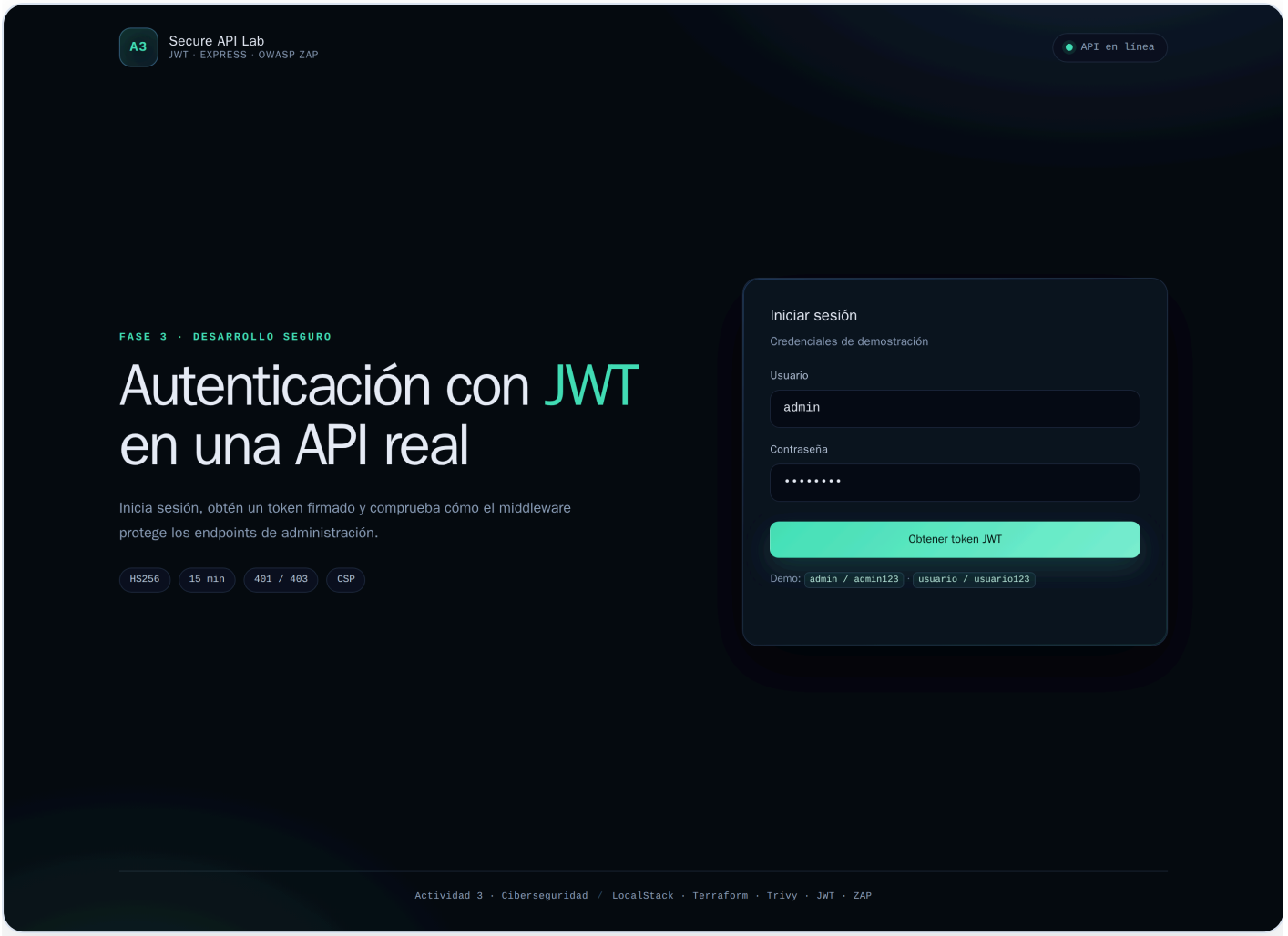
FROM node:18-alpine
RUN apk upgrade --no-cache \
    && rm -rf /usr/local/lib/node_modules/npm \
              /usr/local/lib/node_modules/corepack \
              /usr/local/bin/npm /usr/local/bin/npx /usr/local/bin/corepack
WORKDIR /app
COPY --from=deps /app/node_modules ./node_modules
COPY package.json server.js ./
ENV NODE_ENV=production
USER node
CMD ["node", "server.js"]
```

> La actividad pide `node:18`. A noviembre de 2026 Node 18 ya está fuera de soporte; para un proyecto real se recomienda `node:22-alpine` o superior.

> Limitación del escaneo: Trivy reportó `0` críticas/altas, pero también advirtió que la base Alpine 3.21 ya no recibe actualizaciones de seguridad. El resultado es válido para la base y el estado actual de la base de datos de Trivy, pero no garantiza ausencia de vulnerabilidades futuras. En producción se debe usar una base soportada y reescanear en CI.

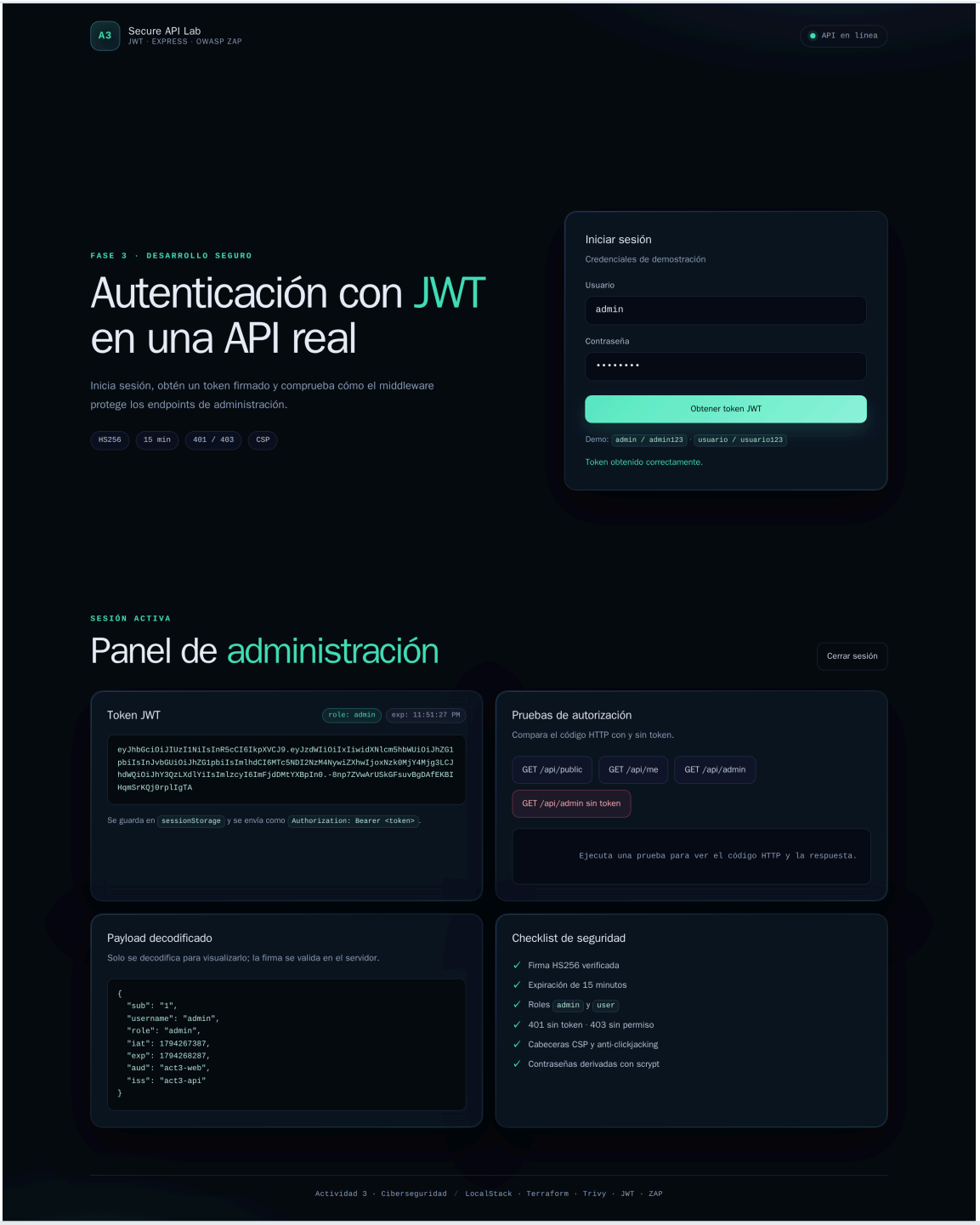
Fase 3 · Sitio y API con JWT

Pantalla de acceso



Login. Formulario servido por Express con CSP, sin `X-Powered-By` y credenciales de demostración visibles.

Panel tras autenticarse

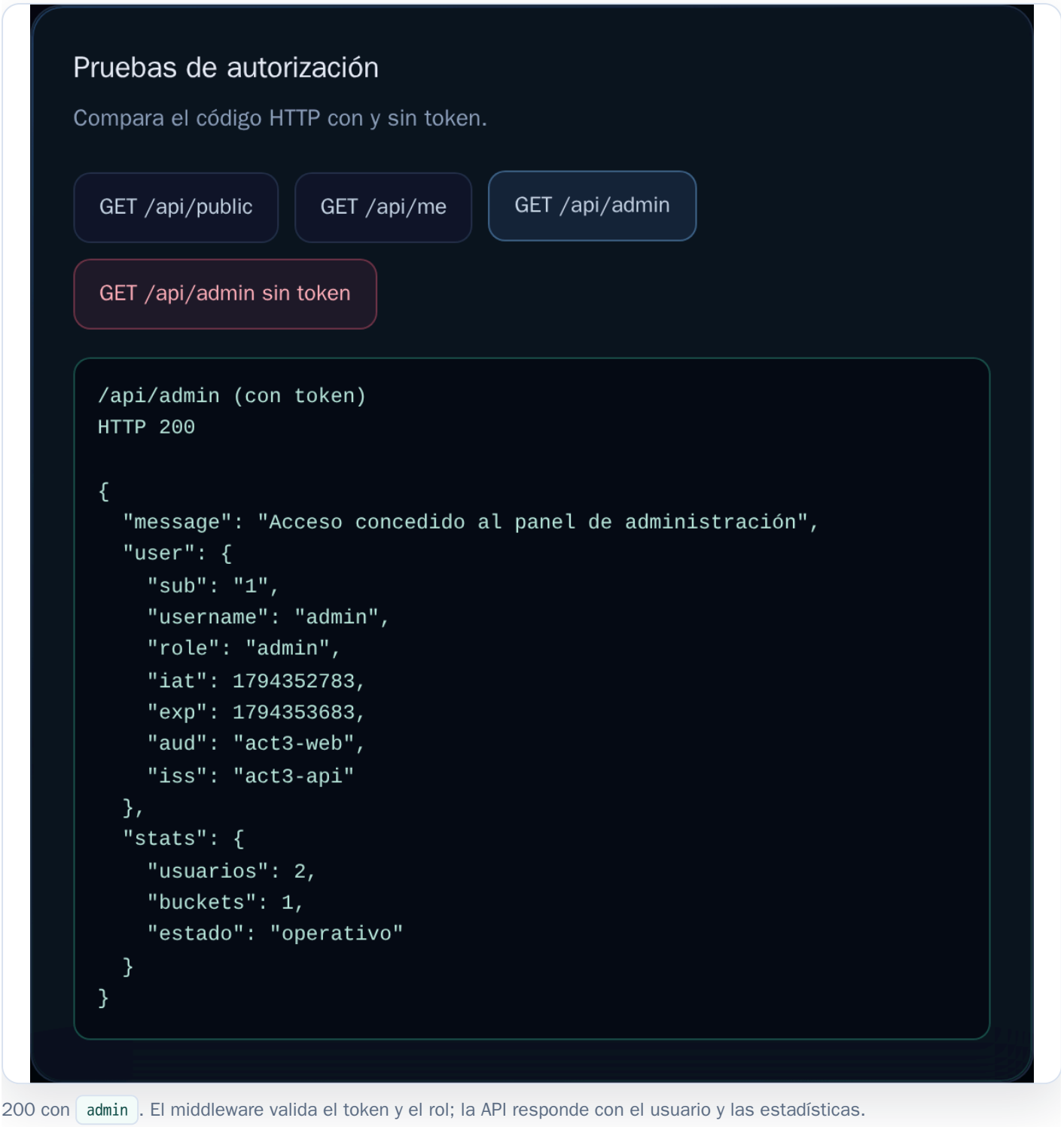


Panel. Token firmado, payload decodificado, botones de prueba y checklist de seguridad.

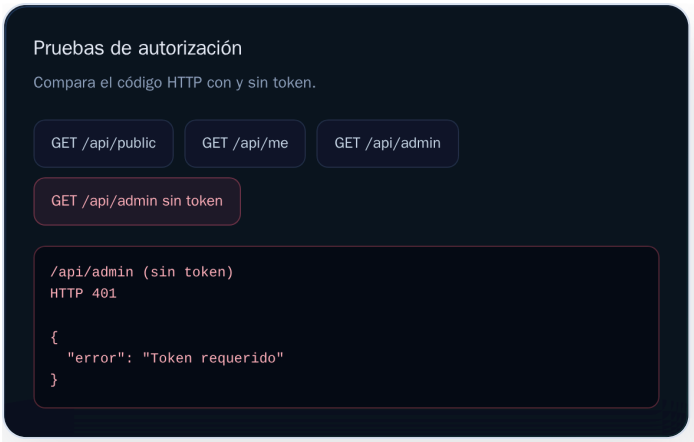
Matriz de autorización

Prueba	Código	Resultado
`GET /api/admin` sin token	401	Correcto
`GET /api/admin` con token inválido	401	Correcto
`GET /api/admin` con token de `usuario`	403	Correcto
`GET /api/admin` con token de `admin`	200	Correcto

Resultado en pantalla



200 con `admin`. El middleware valida el token y el rol; la API responde con el usuario y las estadísticas.



401 sin token. `Token requerido`: la petición no llega al controlador.



403 con rol `user`. Token válido, permisos insuficientes.

Pruebas con curl



curl. Misma matriz comprobada fuera del navegador. Extracto de `fase3/evidencias/pruebas-autorizacion.txt`.

Middleware JWT

```
function authenticate(req, res, next) {
  const authHeader = req.headers.authorization;

  if (!authHeader || !authHeader.startsWith('Bearer ')) {
    return res.status(401).json({ error: 'Token requerido' });
  }

  const token = authHeader.slice(7).trim();

  try {
    req.user = jwt.verify(token, JWT_SECRET, {
      algorithms: ['HS256'],
      issuer: 'act3-api',
      audience: 'act3-web'
    });
    return next();
  } catch (error) {
    return res.status(401).json({ error: 'Token inválido o expirado' });
  }
}

function requireRole(role) {
  return (req, res, next) => {
    if (!req.user || req.user.role !== role) {
      return res.status(403).json({ error: 'Acceso denegado' });
    }
    return next();
  };
}
```

Errores de acceso



Credenciales inválidas. El servidor responde `401` sin revelar si el usuario existe.

Vista móvil



390 px. El diseño se adapta al móvil.

OWASP ZAP

Métrica	Valor
FAIL	0
WARN	3
PASS	65

Las 3 advertencias son de bajo riesgo: contenido no almacenable, formulario de login detectado (esperado) y ausencia de token anti-CSRF. La API usa JSON + `Authorization: Bearer`, por lo que el riesgo CSRF es bajo. Se documenta como mejora.

ZAP Scanning Report

Site: http://localhost:3000

Generated on Mon, 9 Nov 2026 23:38:15

ZAP Version: 2.17.0

ZAP by Checkmarx

Summary of Alerts

Risk Level	Number of Alerts
High	0
Medium	1
Low	0
Informational	4
False Positives:	0

Insights

Level	Reason	Site	Description	Statistic
Info	Informational	http://localhost:3000	Percentage of responses with status code 2xx	77 %
Info	Informational	http://localhost:3000	Percentage of responses with status code 4xx	22 %

Resumen del escaneo. Sin alertas de riesgo alto; 3 de riesgo medio. Informe real de ZAP 2.17.0 sobre `http://localhost:3000`.

Name	Risk Level	Number of Instances
Absence of Anti-CSRF Tokens	Medium	2
Authentication Request Identified	Informational	1
Non-Storable Content	Informational	1
Storable and Cacheable Content	Informational	1
Storable but Non-Cacheable Content	Informational	Systemic

Alertas. Recuento por riesgo e instancias detectadas.

Alert Detail

Medium	Absence of Anti-CSRF Tokens
Description	No Anti-CSRF tokens were found in a HTML submission form. A cross-site request forgery is an attack that involves forcing a victim to send an HTTP request to a target destination without their knowledge or intent in order to perform an action as the victim. The underlying cause is application functionality using predictable URL/form actions in a repeatable way. The nature of the attack is that CSRF exploits the trust that a web site has for a user. By contrast, cross-site scripting (XSS) exploits the trust that a user has for a web site. Like XSS, CSRF attacks are not necessarily cross-site, but they can be. Cross-site request forgery is also known as CSRF, XSRF, one-click attack, session riding, confused deputy, and sea surf. CSRF attacks are effective in a number of situations, including: * The victim has an active session on the target site. * The victim is authenticated via HTTP auth on the target site. * The victim is on the same local network as the target site. CSRF has primarily been used to perform an action against a target site using the victim's privileges, but recent techniques have been discovered to disclose information by gaining access to the response. The risk of information disclosure is dramatically increased when the target site is vulnerable to XSS, because XSS

Detalle. Ejemplo de una de las advertencias y su descripción técnica.

Política IAM de mínimo privilegio

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "LeerObjetosDelBucket",
      "Effect": "Allow",
      "Action": ["s3:GetObject"],
      "Resource": "arn:aws:s3:::actividad3-seguro-local/*"
    },
    {
      "Sid": "ListarSoloEseBucket",
      "Effect": "Allow",
      "Action": ["s3:ListBucket"],
      "Resource": "arn:aws:s3:::actividad3-seguro-local"
    }
  ]
}
```

Conclusiones

Aprendizaje	Aplicación
IaC permite repetir y auditar la infraestructura	Terraform declara bucket, cifrado y bloqueos
La configuración declarada no siempre es la efectiva	Se verificó con AWS CLI y se documentó la limitación de LocalStack
Las imágenes obsoletas acumulan vulnerabilidades	`node:14` tenía 595 hallazgos; la imagen corregida, 0
Autenticación no es lo mismo que autorización	`401` sin token, `403` con token sin rol
El mínimo privilegio reduce el impacto	IAM solo permite `s3:GetObject` y `s3:ListBucket`
Las pruebas automatizadas complementan el análisis	ZAP: 0 FAIL; pruebas manuales: 401/403/200

Referencias

- Aqua Security. (2025). *Trivy: Simple and Comprehensive Vulnerability Scanner for Containers*. https://trivy.dev
- Docker. (2025). *Docker: Accelerated Container Application Development*. https://www.docker.com
- HashiCorp. (s.f.). *Install Terraform*. https://developer.hashicorp.com/terraform/install
- LocalStack. (s.f.). *LocalStack Documentation*. https://docs.localstack.cloud
- OWASP Foundation. (2025). *ZAP: Zed Attack Proxy*. https://www.zaproxy.org

Evidencias y anexos

Anexo	Ubicación
Scripts Terraform	`fase1/`
Reportes Trivy	`fase2/evidencias/`
Código de la API	`fase3/server.js`
Política IAM	`fase3/iam-policy.json`
Diagrama editable	`docs/arquitectura.drawio`
Capturas del sitio y de las herramientas	`docs/capturas/` (19 imágenes)
Repositorio público verificado	github.com/Hugoatpxndx/actividad3-ciberseguridad